

Complementary Branching for #SAT

Bounded-Degree Bounds and Faster #3-SAT

Konstantin Kutzkov

IJCAI 2026

Archon Labs

Propositional Model Counting

CNF formula $F = C_1 \wedge \cdots \wedge C_m$ over n variables, each clause C_i a disjunction of literals.

#SAT: given a Boolean formula F , compute

$$\#SAT(F) = |\{ \sigma : \text{var}(F) \rightarrow \{0, 1\} \mid \sigma \models F \}|.$$

#P-complete [Valiant '79]. **Exact branching algorithms** recursively split F and combine the counts, as in DPLL: $\#SAT(F) = \#SAT(F[x=0]) + \#SAT(F[x=1])$.

Running example

$$F = \underbrace{(\bar{a} \vee x_1 \vee x_2)}_{C_1} \wedge \underbrace{(\bar{a} \vee x_3 \vee x_4)}_{C_2} \wedge \underbrace{(a \vee x_1 \vee y)}_{C_3} \wedge \underbrace{(x_2 \vee \bar{x}_3)}_{C_4} \wedge \underbrace{(\bar{x}_1 \vee x_4 \vee y)}_{C_5}$$

$$n = 6, \quad m = 5, \quad \#SAT(F) = 28, \quad \deg(a) = 3 \text{ and } x_1 \text{ occurs with both } a \text{ and } \bar{a}.$$

Complement Counting

The identity

$$\#SAT(G) = \#SAT(G \wedge H) + \#SAT(G \wedge \neg H) \implies$$

$$\boxed{\#SAT(G \wedge H) = \#SAT(G) - \#SAT(G \wedge \neg H)}$$

for any CNFs G and H over the same set of variables.

Instead of *satisfying* a constraint H , count without it and **subtract the violators**.

- $\#SAT(G)$ is a *relaxed* instance — one constraint fewer.
- $\neg H$ turns a CNF constraint into a DNF one, which we can branch on.
- Only a disjoint partition of assignments is used $\Rightarrow$ **weighted** counting comes for free.

Two ways to use it: take H to be a single *clause*, or some subformula.

Application 1: Clause Branching

$$F : C_1=(\bar{a} \vee x_1 \vee x_2), C_2=(\bar{a} \vee x_3 \vee x_4), C_3=(a \vee x_1 \vee y), C_4=(x_2 \vee \bar{x}_3), C_5=(\bar{x}_1 \vee x_4 \vee y)$$

Take $H = C$, a single clause, so $F = G \wedge C$:

$$\#SAT(F) = \underbrace{\#SAT(G)}_{\text{remove } C} - \underbrace{\#SAT(G \wedge \neg C)}_{\text{falsify } C}, \quad \neg C = \bar{\ell}_1 \wedge \cdots \wedge \bar{\ell}_{|C|}.$$

- **Remove C :** one clause fewer, no variable assigned.
- **Falsify C :** a conjunction of *unit* literals — assigns $|C|$ variables at once.
- **Base case:** no clauses over n free variables $\Rightarrow 2^n$.

► $C = C_4 = (x_2 \vee \bar{x}_3)$, so $G = C_1 \wedge C_2 \wedge C_3 \wedge C_5$ and $\neg C_4 = \bar{x}_2 \wedge x_3$:

$$\underbrace{28}_{\#SAT(F)} = \underbrace{36}_{\#SAT(G)} - \underbrace{8}_{\#SAT(G \wedge \neg C_4)}.$$

Proposition 1

#SAT can be solved in time $\mathcal{O}^(2^m)$ and polynomial space.*

Branching on a Variable

$$F : C_1=(\bar{a} \vee x_1 \vee x_2), C_2=(\bar{a} \vee x_3 \vee x_4), C_3=(a \vee x_1 \vee y), C_4=(x_2 \vee \bar{x}_3), C_5=(\bar{x}_1 \vee x_4 \vee y)$$

Split F at the variable a into the clauses containing a and the rest:

$$F = \underbrace{C_1 \wedge C_2 \wedge C_3}_{F_a} \wedge \underbrace{(x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_4 \vee y)}_{F_{-a}}, \quad a \notin \text{var}(F_{-a}).$$

Setting a satisfies some clauses of F_a and shortens the others, so *each branch is F_{-a} together with a CNF H* :

$$F[a=1] = F_{-a} \wedge \underbrace{(x_1 \vee x_2) \wedge (x_3 \vee x_4)}_{H_1 = F_a[a=1]}$$

$$F[a=0] = F_{-a} \wedge \underbrace{(x_1 \vee y)}_{H_0 = F_a[a=0]}$$

DPLL now recurses on both. But the two branches differ only in H — and complement counting turns each $\#SAT(F_{-a} \wedge H)$ into the *shared* instance F_{-a} minus a correction.

Application 2: DNF Branching

$$F : C_1 = (\bar{a} \vee x_1 \vee x_2), C_2 = (\bar{a} \vee x_3 \vee x_4), C_3 = (a \vee x_1 \vee y), C_4 = (x_2 \vee \bar{x}_3), C_5 = (\bar{x}_1 \vee x_4 \vee y)$$

Apply complement counting to each branch,

$$\#SAT(F_{-a} \wedge H) = \#SAT(F_{-a}) - \#SAT(F_{-a} \wedge \neg H):$$

$$\#SAT(F) = 2 \#SAT(F_{-a}) - \#SAT(F_{-a} \wedge \neg H_0) - \#SAT(F_{-a} \wedge \neg H_1).$$

- $\neg H_0 = \bar{x}_1 \wedge \bar{y}$ — units, assigns 2 more variables.
- $\neg H_1 = \neg((x_1 \vee x_2) \wedge (x_3 \vee x_4))$ is a *DNF*. Splitting it by the first violated clause keeps the branches disjoint, so the counts add:

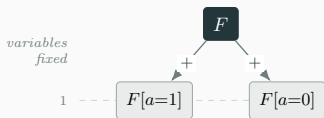
$$\neg(D_1 \wedge D_2) \equiv \neg D_1 \vee (D_1 \wedge \neg D_2).$$

One subproblem costs only a ; the other three fix 3 variables each: $(1, 3, 3, 3)$.

Variable Branching vs. DNF Branching

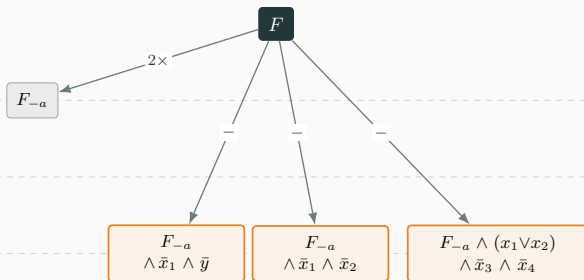
$$F = \underbrace{(\bar{a} \vee x_1 \vee x_2) \wedge (\bar{a} \vee x_3 \vee x_4) \wedge (a \vee x_1 \vee y)}_{F_a : \text{clauses containing } a} \wedge \underbrace{(x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_4 \vee y)}_{F_{-a}}$$

Variable branching on a



2 branches,
each fixing 1 variable

DNF branching on a



4 branches: one fixes 1 variable, three fix 3 each

Instances of Bounded Degree

Either F has a short clause and we branch on it, a $(1, 2, \dots, k)$ -branching for a clause of length k , or all clauses are long — and then a variable of low degree occurs only in long clauses, where DNF branching fixes many variables at once. Either way we always obtain a strong branching.

Theorem 1

The number of satisfying assignments of a CNF formula over n variables of maximum variable degree Δ can be determined in time $\mathcal{O}^(2^{(1-\frac{1}{3D})n})$, where $D = \max(\Delta, 3)$.*

Application 3: Clause Learning

$F : C_1=(\bar{a} \vee x_1 \vee x_2), C_2=(\bar{a} \vee x_3 \vee x_4), C_3=(a \vee x_1 \vee y), C_4=(x_2 \vee \bar{x}_3), C_5=(\bar{x}_1 \vee x_4 \vee y)$

The variable y is *pure*: it occurs only positively, so the branch $F[y=1] = F_{-y}$ carries no new constraint. Splitting F_{-y} by $G = F_y[y=0] = (a \vee x_1) \wedge (\bar{x}_1 \vee x_4)$ gives

$$\#SAT(F) = 2 \#SAT(F_{-y} \wedge G) + \#SAT(F_{-y} \wedge \neg G).$$

Again $\neg G$ is a *DNF* — but a DNF also converts back into a CNF. Instead of branching on it, we simply **learn new clauses**:

$$\neg G \equiv (\bar{a} \vee x_1) \wedge (\bar{a} \vee \bar{x}_4) \wedge (\bar{x}_1 \vee \bar{x}_4).$$

The conversion can blow up, but *low degree* yields few, short clauses.

Theorem 2

Applied to the tight cases of my #3-SAT algorithm [Kutzkov, 2007], this gives a new record for deterministic exact model counting of 3-CNFs: #3-SAT in time $\mathcal{O}^(1.637^n)$.*

Thank you!

A #SAT solver based on complementary counting is currently being developed,
please check

<https://github.com/konstantinkutzhov/COCOA>

for the latest version.